Machine Learning - Assignment 2

Zhenrui Zheng

Chinese University of Hong Kong, Shenzhen
225040512@link.cuhk.edu.cn

Contents

Problem 1: Backpropagation for a MLP **1**

Problem 2: Gradient derivation for two-layer neural network **3**

Problem 3: Convolutional Neural Network **6**

Problem 4: CNNs and Vision Transformers **7**

Problem 5: Using Backpropagation for GANs **8**

Programming I **9**

Programming II: Attention Mechanism **10**

Problem 1: Backpropagation for a MLP

Given a MLP defined as follows:

$$
\begin{aligned}
&\boldsymbol{x} \in \mathbb{R}^D \\
&\boldsymbol{z} = \mathbf{W}\boldsymbol{x} + \boldsymbol{b}_1 \in \mathbb{R}^K \\
&\boldsymbol{h} = \mathrm{ReLU}(\boldsymbol{z}) \in \mathbb{R}^K \\
&\boldsymbol{a} = \mathbf{V}\boldsymbol{h} + \boldsymbol{b}_2 \in \mathbb{R}^C \\
&\mathcal{L} = \mathrm{CrossEntropy}(\boldsymbol{y}, \mathcal{S}(\boldsymbol{a})) \in \mathbb{R}
\end{aligned}
$$

and the gradients of the loss with respect to logits a and hidden z:

$$
\begin{aligned}
\boldsymbol{u}_2 &= \nabla_{\boldsymbol{a}}\mathcal{L} = (\boldsymbol{p} - \boldsymbol{y}) \in \mathbb{R}^C \\
\boldsymbol{u}_1 &= \nabla_{\boldsymbol{z}}\mathcal{L} = (\mathbf{V}^T\boldsymbol{u}_2) \odot H(\boldsymbol{z}) \in \mathbb{R}^K
\end{aligned}
$$

Then, we can derive the gradients for the parameters $\mathbf{V}, \boldsymbol{b}_2, \mathbf{W}, \boldsymbol{b}_1$ and the input $\boldsymbol{x}$ using the chain rule (Hereafter we always use the numerator layout):

$$
\nabla_{\mathbf{V}}\mathcal{L} = \left[\frac{\partial \mathcal{L}}{\partial \mathbf{V}}\right]_{1,:} = \left[\frac{\partial \mathcal{L}}{\partial \boldsymbol{a}}\frac{\partial \boldsymbol{a}}{\partial \mathbf{V}}\right]_{1,:}
$$

Where we know $\partial\mathcal{L}/\partial\boldsymbol{a} = \boldsymbol{u}_2^\top \in \mathbb{R}^{1\times C}$. For $\partial\boldsymbol{a}/\partial\mathbf{V}$, we can expand it according to definition:

$$
\frac{\partial \boldsymbol{a}}{\partial \mathbf{V}} = \begin{pmatrix} \frac{\partial a_1}{\partial \mathbf{V}} \\ \vdots \\ \frac{\partial a_C}{\partial \mathbf{V}} \end{pmatrix} \in \mathbb{R}^{C\times(C\times K)}
$$

Note that $a_c = \sum_{k=1}^K V_{ck}h_k + b_{2c}$, thus:

$$
\frac{\partial a_c}{\partial V_{ij}} = \sum_{k=1}^K \frac{\partial}{\partial V_{ij}} V_{ck}h_k = \sum_{k=1}^K h_k \mathbb{I}(i = c, j = k)
$$

Thus,

$$
\frac{\partial \boldsymbol{a}}{\partial V_{ij}} = \begin{pmatrix} 0 & \cdots & 0 & h_j & 0 & \cdots & 0 \end{pmatrix}^\top \in \mathbb{R}^{C\times 1}
$$

Where the non-zero element is at the i-th position. Thus,

$$
\boldsymbol{u}_2^\top \frac{\partial \boldsymbol{a}}{\partial V_{ij}} = u_{2i}h_j
$$

Thus,

$$
\left[\frac{\partial \mathcal{L}}{\partial \boldsymbol{a}}\frac{\partial \boldsymbol{a}}{\partial \mathbf{V}}\right]_{1,:} = \left[\boldsymbol{u}_2^\top \frac{\partial \boldsymbol{a}}{\partial \mathbf{V}}\right]_{1,:} = \boldsymbol{u}_2\boldsymbol{h}^\top \in \mathbb{R}^{C\times K}
$$

We can do exactly the same thing for $\mathbf{W}$:

$$\nabla_{\mathbf{W}}\mathcal{L} = \left[\frac{\partial \mathcal{L}}{\partial \mathbf{W}}\right]_{1,:} = \left[\frac{\partial \mathcal{L}}{\partial \boldsymbol{z}}\frac{\partial \boldsymbol{z}}{\partial \mathbf{W}}\right]_{1,:} = \left[\boldsymbol{u}_1^\top \frac{\partial \boldsymbol{z}}{\partial \mathbf{W}}\right]_{1,:} = \boldsymbol{u}_1\boldsymbol{x}^\top \in \mathbb{R}^{K\times D}$$

This is because $\boldsymbol{z} = \mathbf{W}\boldsymbol{x} + \boldsymbol{b}_1$ and $\boldsymbol{a} = \mathbf{V}\boldsymbol{h} + \boldsymbol{b}_2$ are identical in form.

For $\boldsymbol{b}_1$ and $\boldsymbol{b}_2$, the derivation would be:

$$\nabla_{\boldsymbol{b}_2}\mathcal{L} = \left(\frac{\partial \mathcal{L}}{\partial \boldsymbol{b}_2}\right)^\top = \left(\frac{\partial \mathcal{L}}{\partial \boldsymbol{a}}\frac{\partial \boldsymbol{a}}{\partial \boldsymbol{b}_2}\right)^\top = \boldsymbol{u}_2 \in \mathbb{R}^C$$

Where $\partial \boldsymbol{a}/\partial \boldsymbol{b}_2 = \mathbb{I} \in \mathbb{R}^{C\times C}$. We can do exactly the same for $\boldsymbol{b}_1$ and get $\nabla_{\boldsymbol{b}_1}\mathcal{L} = \boldsymbol{u}_1 \in \mathbb{R}^C$.

For $\boldsymbol{x}$, the derivation would be:

$$\nabla_{\boldsymbol{x}}\mathcal{L} = \left(\frac{\partial \mathcal{L}}{\partial \boldsymbol{x}}\right)^\top = \left(\frac{\partial \mathcal{L}}{\partial \boldsymbol{z}}\frac{\partial \boldsymbol{z}}{\partial \boldsymbol{x}}\right)^\top = \left(\boldsymbol{u}_1^\top \mathbf{W}\right)^\top = \mathbf{W}^\top \boldsymbol{u}_1 \in \mathbb{R}^D$$

Where

$$\frac{\partial \boldsymbol{z}}{\partial \boldsymbol{x}} = \begin{pmatrix} \frac{\partial z_1}{\partial x_1} & \cdots & \frac{\partial z_1}{\partial x_D} \\ \vdots & \ddots & \vdots \\ \frac{\partial z_K}{\partial x_1} & \cdots & \frac{\partial z_K}{\partial x_D} \end{pmatrix} = \mathbf{W}$$

Because

$$\frac{\partial z_i}{\partial x_j} = \frac{\partial}{\partial x_j}\left(\sum_{k=1}^{K} W_{ik}x_k + b_{1i}\right) = W_{ij}$$

Problem 2: Gradient derivation for two-layer neural network

(a) Gradient of E wrt $\boldsymbol{z}_2$

Use the chain rule:

$$\frac{\partial E}{\partial \boldsymbol{z}_2} = \frac{\partial E}{\partial \hat{\boldsymbol{o}}} \frac{\partial \hat{\boldsymbol{o}}}{\partial \boldsymbol{z}_2}$$

Which can be expanded as:

$$\frac{\partial E}{\partial (\boldsymbol{z}_2)_j} = \sum_{i=1}^{K} \frac{\partial E}{\partial (\hat{\boldsymbol{o}}_i)} \frac{\partial (\hat{\boldsymbol{o}}_i)}{\partial (\boldsymbol{z}_2)_j}$$

Where the derivative of E wrt to $\hat{\boldsymbol{o}}_i$ is:

$$\frac{\partial E}{\partial (\hat{\boldsymbol{o}}_i)} = \frac{\partial}{\partial (\hat{\boldsymbol{o}}_i)} \left(-\sum_{k=1}^{K} \boldsymbol{o}_k \log(\hat{\boldsymbol{o}}_k) \right) = -\frac{\boldsymbol{o}_i}{\hat{\boldsymbol{o}}_i}$$

And the derivative of $\hat{\boldsymbol{o}}_i$ wrt to $(\boldsymbol{z}_2)_j$ is:

$$\frac{\partial (\hat{\boldsymbol{o}}_i)}{\partial (\boldsymbol{z}_2)_j} = \frac{\partial}{\partial (\boldsymbol{z}_2)_j} (\text{softmax}(\boldsymbol{z}_2))_i = \frac{\partial}{\partial (\boldsymbol{z}_2)_j} \left(\frac{\exp((\boldsymbol{z}_2)_i)}{\sum_{k=1}^{K} \exp((\boldsymbol{z}_2)_k)} \right) = \hat{\boldsymbol{o}}_i (\mathbb{I}(i=j) - \hat{\boldsymbol{o}}_j)$$

Thus,

$$\begin{aligned}
\frac{\partial E}{\partial (\boldsymbol{z}_2)_j} &= \sum_{i=1}^{K} -\frac{\boldsymbol{o}_i}{\hat{\boldsymbol{o}}_i} \hat{\boldsymbol{o}}_i (\mathbb{I}(i=j) - \hat{\boldsymbol{o}}_j) \\
&= -\boldsymbol{o}_j (1 - \hat{\boldsymbol{o}}_j) + \sum_{i \neq j} \boldsymbol{o}_i \hat{\boldsymbol{o}}_j \\
&= -\boldsymbol{o}_j + \hat{\boldsymbol{o}}_j \sum_{i=1}^{K} \boldsymbol{o}_i \\
&= \hat{\boldsymbol{o}}_j - \boldsymbol{o}_j \quad (\text{Since} \sum_i \boldsymbol{o}_i = 1)
\end{aligned}$$

Thus, the vector form is:

$$\frac{\partial E}{\partial \boldsymbol{z}_2} = \begin{pmatrix} \frac{\partial E}{\partial (\boldsymbol{z}_2)_1} \\ \vdots \\ \frac{\partial E}{\partial (\boldsymbol{z}_2)_K} \end{pmatrix} = \begin{pmatrix} \hat{\boldsymbol{o}}_1 - \boldsymbol{o}_1 \\ \vdots \\ \hat{\boldsymbol{o}}_K - \boldsymbol{o}_K \end{pmatrix} = \hat{\boldsymbol{o}} - \boldsymbol{o}$$

(b) Gradient of E wrt $\mathbf{W}_2$ and $\boldsymbol{b}_2$

Similarly, we can use the chain rule:

$$\frac{\partial E}{\partial \mathbf{W}_2} = \frac{\partial E}{\partial \boldsymbol{z}_2} \frac{\partial \boldsymbol{z}_2}{\partial \mathbf{W}_2}$$

Which can be expanded as:

$$\frac{\partial E}{\partial (\mathbf{W}_2)_{ij}} = \sum_{k=1}^{K} \frac{\partial E}{\partial (\boldsymbol{z}_2)_k} \frac{\partial (\boldsymbol{z}_2)_k}{\partial (\mathbf{W}_2)_{ij}}$$

Where the derivative of $\boldsymbol{z}_2$ wrt to $(\mathbf{W}_2)_{ij}$ is:

$$\frac{\partial (\boldsymbol{z}_2)_k}{\partial (\mathbf{W}_2)_{ij}} = \frac{\partial}{\partial (\mathbf{W}_2)_{ij}} \left(\sum_{l=1}^{d} (\mathbf{W}_2)_{kl} \boldsymbol{h}_l + \boldsymbol{b_2}_k \right) = \boldsymbol{h}_j \mathbb{I}(k = i, l = j)$$

Thus,

$$\begin{aligned} \frac{\partial E}{\partial (\mathbf{W}_2)_{ij}} &= \sum_{k=1}^{K} (\hat{\boldsymbol{o}}_k - \boldsymbol{o}_k) \boldsymbol{h}_j \mathbb{I}(k = i, l = j) \\ &= (\hat{\boldsymbol{o}}_i - \boldsymbol{o}_i) \boldsymbol{h}_j \\ \frac{\partial E}{\partial \mathbf{W}_2} &= (\hat{\boldsymbol{o}} - \boldsymbol{o}) \boldsymbol{h}^\top \end{aligned}$$

For $\boldsymbol{b}_2$, the derivation would be:

$$\frac{\partial E}{\partial \boldsymbol{b}_2} = \frac{\partial E}{\partial \boldsymbol{z}_2} \frac{\partial \boldsymbol{z}_2}{\partial \boldsymbol{b}_2}$$

Which can be expanded as:

$$\frac{\partial E}{\partial (\boldsymbol{b}_2)_j} = \sum_{k=1}^{K} \frac{\partial E}{\partial (\boldsymbol{z}_2)_k} \frac{\partial (\boldsymbol{z}_2)_k}{\partial (\boldsymbol{b}_2)_j}$$

Where the derivative of $\boldsymbol{z}_2$ wrt to $(\boldsymbol{b}_2)_j$ is:

$$\frac{\partial (\boldsymbol{z}_2)_k}{\partial (\boldsymbol{b}_2)_j} = \frac{\partial}{\partial (\boldsymbol{b}_2)_j} \left(\sum_{l=1}^{d} (\mathbf{W}_2)_{kl} \boldsymbol{h}_l + \boldsymbol{b_2}_k \right) = \mathbb{I}(k = j)$$

Thus,

$$\begin{aligned} \frac{\partial E}{\partial (\boldsymbol{b}_2)_j} &= \sum_{k=1}^{K} (\hat{\boldsymbol{o}}_k - \boldsymbol{o}_k) \mathbb{I}(k = j) \\ &= (\hat{\boldsymbol{o}}_j - \boldsymbol{o}_j) \\ \frac{\partial E}{\partial \boldsymbol{b}_2} &= (\hat{\boldsymbol{o}} - \boldsymbol{o}) \end{aligned}$$

(c) Gradient of E wrt $\boldsymbol{h}$ and $\boldsymbol{z}_1$

Similarly:

$$\frac{\partial E}{\partial \boldsymbol{h}} = \frac{\partial E}{\partial \boldsymbol{z}_2} \frac{\partial \boldsymbol{z}_2}{\partial \boldsymbol{h}}$$

Which can be expanded as:

$$\frac{\partial E}{\partial (\boldsymbol{h})_j} = \sum_{k=1}^{K} \frac{\partial E}{\partial (\boldsymbol{z}_2)_k} \frac{\partial (\boldsymbol{z}_2)_k}{\partial (\boldsymbol{h})_j}$$

Where the derivative of $\boldsymbol{z}_2$ wrt to $(\boldsymbol{h})_j$ is:

$$\frac{\partial(\boldsymbol{z}_2)_k}{\partial(\boldsymbol{h})_j} = \frac{\partial}{\partial(\boldsymbol{h})_j}\left(\sum_{l=1}^{d}(\mathbf{W}_2)_{kl}\boldsymbol{h}_l + \boldsymbol{b}_{\mathbf{2}k}\right) = (\mathbf{W}_2)_{kj}$$

Thus,

$$\frac{\partial E}{\partial(\boldsymbol{h})_j} = \sum_{k=1}^{K}(\hat{\boldsymbol{o}}_k - \boldsymbol{o}_k)(\mathbf{W}_2)_{kj}$$

$$\frac{\partial E}{\partial \boldsymbol{h}} = \mathbf{W}_2^\top(\hat{\boldsymbol{o}} - \boldsymbol{o})$$

For $\boldsymbol{z}_1$, the derivation would be:

$$\frac{\partial E}{\partial \boldsymbol{z}_1} = \frac{\partial E}{\partial \boldsymbol{h}}\frac{\partial \boldsymbol{h}}{\partial \boldsymbol{z}_1} = \frac{\partial E}{\partial \boldsymbol{h}} \odot \mathrm{ReLU}'(\boldsymbol{z}_1) = \mathbf{W}_1^\top(\hat{\boldsymbol{o}} - \boldsymbol{o}) \odot \mathbb{I}(\boldsymbol{z}_1 > 0)$$

(d) Gradient of E wrt $\boldsymbol{W}_1$ and $\boldsymbol{b}_1$

Similarly:

$$\frac{\partial E}{\partial \mathbf{W}_1} = \frac{\partial E}{\partial \boldsymbol{z}_1}\frac{\partial \boldsymbol{z}_1}{\partial \mathbf{W}_1}$$

Which can be expanded as:

$$\frac{\partial E}{\partial(\boldsymbol{W}_1)_{ij}} = \sum_{k=1}^{d}\frac{\partial E}{\partial(\boldsymbol{z}_1)_k}\frac{\partial(\boldsymbol{z}_1)_k}{\partial(\boldsymbol{W}_1)_{ij}}$$

Where the derivative of $\boldsymbol{z}_1$ wrt to $(\boldsymbol{W}_1)_{ij}$ is:

$$\frac{\partial(\boldsymbol{z}_1)_k}{\partial(\boldsymbol{W}_1)_{ij}} = \frac{\partial}{\partial(\boldsymbol{W}_1)_{ij}}\left(\sum_{l=1}^{n}(\mathbf{W}_1)_{kl}\boldsymbol{x}_l + \boldsymbol{b}_{\mathbf{1}k}\right) = \boldsymbol{x}_j\mathbb{I}(k = i, l = j)$$

Similar to $\partial E/\partial \mathbf{W}_2$, This results in an outer product of the upstream gradient $\frac{\partial E}{\partial \boldsymbol{z}_1}$ and the input to the layer, $\boldsymbol{x}$:

$$\frac{\partial E}{\partial \mathbf{W}_1} = \left[\mathbf{W}_2^\top(\hat{\boldsymbol{o}} - \boldsymbol{o}) \odot \mathbb{I}(\boldsymbol{z}_1 > 0)\right]\boldsymbol{x}^\top$$

For $\boldsymbol{b}_1$, the derivation would be:

$$\frac{\partial E}{\partial \boldsymbol{b}_1} = \frac{\partial E}{\partial \boldsymbol{z}_1}\frac{\partial \boldsymbol{z}_1}{\partial \boldsymbol{b}_1} = \frac{\partial E}{\partial \boldsymbol{z}_1}\mathbf{I} = \mathbf{W}_1^\top(\hat{\boldsymbol{o}} - \boldsymbol{o}) \odot \mathbb{I}(\boldsymbol{z}_1 > 0)$$

Problem 3: Convolutional Neural Network

(a) Activation Volume Dimensions and Number of Parameters

The formula for the output dimension of a convolutional or pooling layer is:

$$O = \lfloor \frac{W_{in} - F + 2P}{S} \rfloor + 1$$

where W_{in} is the input dimension, F is the filter/pool size, P is padding, and S is stride. The number of parameters for each layer type is:

- Convolutional Layer (CONV): $(F \times F \times D_{in} + 1) \times N_{filters}$
- Pooling Layer (POOL): 0 parameters
- Fully-Connected Layer (FC): $(N_{in} + 1) \times N_{out}$

Layer	Activation Volume Dimensions	Number of Parameters
INPUT	$32 \times 32 \times 1$	0
CONV5-10	$28 \times 28 \times 10$	$(5 \times 5 \times 1 + 1) \times 10 = 260$
POOL2	$14 \times 14 \times 10$	0
CONV5-10	$10 \times 10 \times 10$	$(5 \times 5 \times 10 + 1) \times 10 = 2,510$
POOL2	$5 \times 5 \times 10$	0
FC-10	10×1	$(5 \times 5 \times 10 + 1) \times 10 = 2,510$

(b) Reason for using a 1x1 Convolution

Less rigorously, we say convolutional neural networks process input in two ways:

- **spatial operations**, which capture spatial patterns between adjacent pixels, such as edges or textures, through receptive fields;
- **channel operations**, used to identify information such as colors in RGB space.

In the original convolutional neural network architecture, these two were simultaneously processed by the conv layer; however, in "depthwise separable convolution", they are decoupled and such architectures have been proven to be faster and more efficient.

From this perspective, a 1×1 convolution is equivalent to independently processing channel information without changing spatial information, and is often used in conjunction with its complement, i.e., a "receptive field" network that only processes spatial information and does not involve channel information. Intuitively, if two components of a statistical distribution are (to some extent) independent, it will be easier to handle them separately, and the use of 1×1 convolution is consistent with such intuition.

As a result, for a layer with D_{out} output dimensions, the number of parameters required for "depthwise separable convolution" is only about $1/D_{\text{out}}$ of the original, without damaging (or even improving) the performance.

Problem 4: CNNs and Vision Transformers

1. **What are the two main operations used in CNNs to process images?**:
   B. Convolution and pooling.

2. **Why do Vision Transformers (ViTs) need positional encodings?**:
   B. To remember where each image patch came from.

3. **Which architecture typically works better with small datasets?**:
   B. CNN.
   CNN possess strong inductive biases, such as local receptive fields and weight sharing. These biases enable CNN to effectively learn meaningful locality features with a relatively smaller number of parameters. This generally leads to better performance for CNN on small datasets compared to ViT, while the later typically require large amounts of data to learn effective representations and reduce overfitting induced by global attention mechanism. (As an evidence, some follow-up work attempts to add regularization to pixel distance and claims performance improvements.)

4. **Patch Processing: An image of size 224×224 is divided into 16×16 patches for a Vision Transformer. (1) How many patches will there be? (2) If each patch is flattened into a vector, what will be its length for an RGB image?**:
   (1) There will be $(224//16) \times (224//16) = 14 \times 14 = 196$ patches.
   (2) The length of each patch will be $16 \times 16 \times 3 = 768$ for an RGB image.

5. **For each feature below, indicate whether it applies to**:
   (1) Processes image patches in sequential order: (V) or (N)
   Although ViT divides pixels into patches and processes them in a "sequential fashion", it is less appropriate to call it "sequential order" because all patches are simultaneously fed into the attention layer for computation, and there is no logical order between patches. Most ViT are discriminative models, so they are non-causal during training (no causal mask), but some generative ViT are indeed trained causally and are "sequential" during generation.
   (2) Can handle patches in parallel fashion: (B)
   (3) Uses weight sharing across spatial locations: (C) or (B)
   There's no doubt about CNN's spatial weight sharing, but for ViT, this concept is more ambiguous. The weight matrices ($\mathbf{W}_q$, $\mathbf{W}_k$, $\mathbf{W}_v$) for different patches are shared, while the patches are distributed across different spatial locations. Furthermore, within each patch, the parameters of the linear layers are also shared. Although for pixels at certain specific positions in the input, the weights they "see" may not be the same (e.g., within the same patch, the parameters for two different positions are different), and most positional encodings introduce some spatial bias. But at least during training, this heterogeneity can be reduced through spatial augmentation operations or by using true "relative" positional encodings.
   (4) Requires positional encoding of patches: (V)
   (5) Naturally preserves local spatial relationships: (C)
   (6) Naturally preserves global spatial relationships: (V)

Problem 5: Using Backpropagation for GANs

Programming I

Programming II: Attention Mechanism

We implement the scaled dot-product attention mechanism equivalent to the following:

```python
def scaled_dot_product_attention(query, key, value, attn_mask=None, dropout_p=0.0):
    "Compute 'Inefficient Scaled Dot Product Attention'"
    L, S = query.size(-2), key.size(-2)
    scale_factor = 1 / math.sqrt(query.size(-1))
    attn_bias = torch.zeros(L, S, dtype=query.dtype, device=query.device)

    if attn_mask is not None:
        if attn_mask.dtype == torch.bool:
            attn_bias.masked_fill_(attn_mask.logical_not(), float("-inf"))
        else:
            attn_bias = attn_mask + attn_bias

    attn_weight = query @ key.transpose(-2, -1) * scale_factor
    attn_weight += attn_bias
    attn_weight = torch.softmax(attn_weight, dim=-1)
    attn_weight = torch.dropout(attn_weight, dropout_p, train=True)
    return attn_weight @ value
```